RiskMap

Safest-route navigation for New York City · build plan v2

> **What we want:** a Next.js map app where you type where you are and where you're going. Instead of the fastest route, it returns the **safest** one — steering around streets with a history of injury crashes, and around known street-flooding locations when the weather API says heavy rain is hitting that borough. Two routes are drawn side by side so the difference is obvious.

1 · Stack

LAYER	CHOICE	WHY THIS ONE
Frontend	**Next.js** (App Router)	App + API routes in one deploy on Vercel
Database	**Supabase** Postgres + PostGIS	Enable the PostGIS extension — it gives us real distance queries and "snap point to nearest road" in SQL instead of Python
Map	**Mapbox GL JS**	Free tier is 50k loads/month, no billing card. Includes geocoding (address → lat/lon) so we don't need a second API. Google Maps needs a billing account attached — avoid that during a hackathon.
Weather	**Open-Meteo**	No API key, no signup, no rate limit worth worrying about. 5 calls, one per borough.
Routing	**A*** in a Next.js API route	Graph loaded from Supabase and cached in memory
Prep	Python (pandas) → Supabase	One-off scripts to clean and upload. Never runs during the demo.

2 · The three data layers

LAYER	FILE IN REPO	SIZE	STATE
Crashes	`Motor_Vehicle_Collisions…csv`	2,269,187 rows	READY
Flooding	`FloodNet__Street_Flooding…csv`	2,929 events	NEEDS GEOCODING
Crime	`sample_complaints.csv`	20,000 rows	SYNTHETIC — REPLACE

Crashes — the strong layer

2.27M	756,353	3,617
CRASHES 2012–2026	INJURED	KILLED

Has clean `LATITUDE` / `LONGITUDE` on 89% of rows plus injury and fatality counts — everything the risk score needs. **Use 2022 onward only** (413,780 rows): crash volume fell from ~230k/year pre-COVID to ~86k in 2025, so older data describes traffic that no longer exists.

Two traps: 7,591 rows sit at exactly `(0, 0)` — filter by NYC bounding box, not just `NOT NULL`. And 30% of rows have no `BOROUGH`, so filter by coordinates instead of that column.

Flooding — real, but no coordinates

2,929 measured flood events from 294 FloodNet sensors, Nov 2020 → Aug 2026, and it's **accelerating**: 142 events in 2022 versus 1,021 already in 2026. Median peak depth 3.5 inches; 1,295 events reached 4 inches or more.

> **The file has no latitude or longitude column.** Location lives only inside `Sensor Name`, as a borough prefix plus a street: `Q - Beach 84 St`, `BX - Ditmars St/Hunter Ave 2`. Fix: strip the prefix and trailing number, then geocode the 294 unique names once with Mapbox and store the results in Supabase. It's 294 calls — a one-time script, not a runtime cost.

Flooding is highly concentrated, which is good for us: Queens alone accounts for 1,710 of 2,929 events (mostly the Rockaways), then Bronx 485, Brooklyn 425, Staten Island 223, Manhattan 79. A handful of sensors dominate — `Q - Beach 84 St` alone logged 438 floods. That means a small number of avoid-zones covers most of the real risk.

Crime — this file is not real data

> **`sample_complaints.csv` is synthetic and cannot support any map layer.** Four independent checks confirm it:
>
> - Every one of the 20,000 rows has the identical timestamp `12:00:00` — no time-of-day signal exists.
> - All four boroughs have *the same* coordinate distribution (mean 40.767, −73.972 for each). The coordinates are uniform random inside one box and do not correspond to the stated borough.
> - Only 6 offense types, each appearing ~3,300 times — a real distribution is never that flat.
> - Staten Island is missing entirely, and offense-to-severity pairings are wrong (assault 3 is a misdemeanour in reality, not a felony).

What to do: download the real *NYPD Complaint Data Historic* (or *Year to Date*) from NYC Open Data — same column names, so nothing downstream changes. If there's no time for that, **drop the crime layer and ship crashes + flooding**. Two working layers beat three where one is fabricated, and a judge who spots the flat distribution will discount the whole project.

3 · How risk becomes a route

Every hazard collapses to one number per street segment. No model, no training — a weighted sum tuned by eye.

```
cost = length × (1 + α·crash_risk + β·flood_risk·raining)
```

TERM	RANGE	SOURCE
`crash_risk`	0 – 1	Crashes within 50 m of the segment, weighted `1 + 3·injuries + 10·deaths`, then percentile-ranked
`flood_risk`	0 – 1	Segment within 200 m of a FloodNet sensor, scaled by that sensor's event count
`raining`	0 or 1	Open-Meteo says heavy rain in *that segment's borough*
`α, β`	~2 – 5	Hand-tuned sliders. α=3 makes a max-risk street feel 4× longer.

> **Keep the penalty multiplicative.** Risk only ever pushes cost *upward*, so `cost ≥ length` always holds — which keeps straight-line distance a valid A* heuristic and the routes correct. If safe streets were ever made *cheaper* than their true length, A* would return wrong paths and nothing would visibly break.

4 · Weather, per borough

Open-Meteo, five calls — one per borough centroid. No key required.

```
const BOROS = {
  manhattan:     [40.7831, -73.9712],
  brooklyn:      [40.6782, -73.9442],
  queens:        [40.7282, -73.7949],
  bronx:         [40.8448, -73.8648],
  staten_island: [40.5795, -74.1502],
};

GET api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&hourly=precipitation

raining[boro] = max(precipitation, next 3h) > 4mm/h
```

Cache the result in Supabase for 15 minutes so repeated routing requests don't re-hit the API. Five booleans is the entire weather payload.

> **Per-borough is exactly the right resolution** — and it's honest. Rain genuinely varies across a 50 km city, so five readings is a real improvement over one. But it does not vary block to block, so the *geography* of the flood risk has to come from the FloodNet sensors, not the weather API. If a judge asks how granular the weather is, "five boroughs, and the flood sensors supply the street-level detail" is a strong answer.

5 · Supabase schema

```
-- enable once
create extension if not exists postgis;

road_edges          -- the routable graph, precomputed offline
  id, from_node, to_node,
  geom          geography(LineString),
  length_m      float,
  borough       text,
  crash_risk    float,   -- 0-1
  flood_risk    float    -- 0-1

flood_sensors       -- 294 rows, geocoded once
  sensor_name, borough, geom geography(Point),
  event_count int, max_depth_in float

weather_cache       -- 5 rows, refreshed every 15 min
  borough, raining bool, checked_at timestamptz

crashes_raw         -- optional: only if we want a heat-map overlay
  geom geography(Point), injuries int, deaths int, occurred_at timestamptz
```

The two `_risk` columns are written once by the Python prep script. At request time the API route reads `road_edges`, applies the weather booleans, and runs A*. **Nothing heavy happens during the demo.**

6 · Build order

1 — Map + graph ~1 hr

Next.js page with Mapbox, two address inputs using Mapbox geocoding. Load the Manhattan street graph (OSMnx export → `road_edges`) and run plain A* on distance. *No risk data yet* — but this is already a working routing app, so we can never end with nothing.

2 — Crash layer ~2 hrs · the real work

Python: filter 2022+, drop the `(0,0)` rows, snap each crash to the nearest edge with PostGIS, aggregate the injury-weighted score, percentile-rank, write `crash_risk`. Test on 10k rows before running all 414k.

3 — Two-route comparison ~30 min

Run A* twice per request — once on length, once on risk-weighted cost. Draw grey and green lines. **This is where it becomes a demo instead of a map.**

4 — Flood layer ~1 hr

Geocode the 294 sensor names, load into `flood_sensors`, set `flood_risk` on edges within 200 m, add the β term behind a manual rain toggle.

5 — Weather API ~20 min

Replace the toggle with 5 live Open-Meteo calls. **Keep the manual override** — it probably won't be raining during the demo.

6 — Polish whatever's left

Reason string ("avoided 3 high-crash intersections, +4 min"), crash heat-map toggle, and an α slider judges can drag to watch the route get more cautious live.

7 · Demo

1. Enter start and destination. Grey line appears — "this is what Google gives you."
2. Hit *Safe route*. Green line diverges around the high-crash corridors. "+4 minutes, avoids three intersections with 2,000 crashes on record."
3. Flip heavy rain on for Queens. The green route **moves again**, now dodging the Rockaways flood cluster.
4. Drag the α slider up and watch the route grow more cautious in real time.

> **Protect one thing:** finish step 3 early. Crashes + two-route comparison is already a complete, defensible project. Flood and weather make it better — they are not prerequisites for having something that works.

RiskMap build plan v2 · Next.js + Supabase/PostGIS + Mapbox + Open-Meteo · crash data 2,269,187 records (2012-07-01 → 2026-06-11) · FloodNet 2,929 events across 294 sensors